

Computer Science Final Year Project — Standard Outline & Guide

PART A: FINAL YEAR PROJECT (4th Year) — Full Structure

1. Preliminary Pages

Title Page

Project title, student name & matric/reg number, department, faculty, university, supervisor's name, month/year of submission

Declaration Page — statement that the work is original

Certification/Approval Page — supervisor and HOD sign-off

Dedication (optional)

Acknowledgements

Abstract (150–300 words: problem, method, result, conclusion in miniature)

Table of Contents

List of Figures

List of Tables

List of Abbreviations/Acronyms (if applicable)

2. Chapter One — Introduction

1.1 Background of the Study

1.2 Statement of the Problem

1.3 Aim and Objectives

1.4 Research Questions / Hypotheses (if applicable)

1.5 Significance of the Study

1.6 Scope and Limitations of the Study

1.7 Definition of Terms

1.8 Organization of the Report (brief chapter-by-chapter summary)

3. Chapter Two — Literature Review

2.1 Introduction

2.2 Conceptual Framework / Theoretical Background

2.3 Review of Related Concepts (key technologies, algorithms, models)

2.4 Review of Related/Existing Systems

2.5 Comparative Analysis of Existing Systems (table comparing features, strengths, weaknesses)

2.6 Gaps Identified in Literature

2.7 Summary of Literature Review

4. Chapter Three — System Analysis and Methodology

3.1 Introduction

3.2 Research/Development Methodology (e.g., Agile, Waterfall, RAD, OOADM)

3.3 Analysis of the Existing System

3.4 Problems of the Existing System

3.5 Analysis of the Proposed System

3.6 Requirement Analysis

Functional Requirements

Non-Functional Requirements

3.7 System Design Tools (UML diagrams: Use Case, Class, Sequence, Activity, ERD/DFD)

3.8 Hardware and Software Requirements

3.9 Justification for Chosen Tools/Technology Stack

5. Chapter Four — System Design and Implementation

4.1 Introduction

4.2 System Architecture (high-level diagram)

4.3 Database Design (ER diagram, schema, table structures/data dictionary)

4.4 Input/Output Design (mockups, form layouts)

4.5 Program/Module Design (pseudocode, flowcharts, algorithms)

4.6 Implementation Environment (languages, frameworks, IDE, hosting)

4.7 Implementation Details (key code snippets/screenshots explained)

4.8 Testing

Unit Testing

Integration Testing

System/Acceptance Testing

Test Cases and Results (tabulated)

4.9 System Requirements for Deployment (installation, environment setup)

6. Chapter Five — Summary, Conclusion and Recommendations

5.1 Summary of the Study

5.2 Achievements/Contributions of the Project

5.3 Challenges Encountered

5.4 Conclusion

5.5 Recommendations

5.6 Suggestions for Future Work

7. End Matter

References/Bibliography (consistent style: APA, IEEE, or as required by department)

Appendices

Appendix A: Full Source Code (or link to repository)

Appendix B: Sample Screenshots/User Interface

Appendix C: Questionnaires/Interview Guide (if used)

Appendix D: User Manual

Appendix E: Turnitin/Plagiarism Report (if required)

PART B: MINI PROJECT LAYOUT (Third Year Students)

A mini project is shorter and lighter — usually one semester, smaller scope, less formal review, but should still be professionally structured.

Suggested Structure

Title Page — title, name, level/reg number, course code, supervisor, date

Abstract (100–150 words)

Table of Contents

Chapter One — Introduction

1.1 Background

1.2 Problem Statement

1.3 Objectives

1.4 Scope

1.5 Significance

Chapter Two — Literature Review (condensed)

2.1 Related Concepts

2.2 Related Works/Systems (2–4 reviewed, not exhaustive)

Chapter Three — Design and Methodology

3.1 Methodology Used

3.2 System Requirements

3.3 Design Diagrams (at least Use Case + one structural diagram e.g. ERD/Class Diagram)

Chapter Four — Implementation and Results

4.1 Tools Used

4.2 How It Works (with screenshots)

4.3 Testing (basic — a test table is enough)

Chapter Five — Conclusion

5.1 Summary

5.2 Limitations

5.3 Recommendations

References Appendix — Source code snippets, screenshots

Mini projects can skip: extensive literature comparison tables, full data dictionaries, and formal declaration/certification pages, unless your department requires them.

PART C: Practical Guide for Third-Year Students

Pick a scoped problem, not a broad one. "A student attendance system using facial recognition" is scoped; "AI for education" is not.

Confirm feasibility in week 1–2. Check that the tech stack, data, and time available actually match your idea.

Start the literature review early — even 5–6 solid sources give your project credibility and help you avoid duplicating existing work.

Design before you code. Draw your use case diagram and ERD/class diagram before writing implementation code — it saves rework.

Version control from day one. Use Git/GitHub even for a solo mini project; it demonstrates professionalism and protects your work.

Keep a working log. Note what you built each week — this becomes the backbone of Chapter Four and saves you from forgetting details at write-up time.

Test as you build, not at the end. Write at least a simple test table (input → expected output → actual output) per module.

Write incrementally. Don't wait until the deadline to start Chapter One — draft chapters alongside development.

Use consistent formatting. One font (e.g., Times New Roman 12pt or as required), one citation style, numbered headings — this is what markers notice first.

Get supervisor feedback at each chapter, not just at the end — it prevents large-scale revisions later.

Prepare a short demo/defense pitch even for mini projects — it's good practice for the final year defense.

Quick Comparison: Mini Project vs Final Year Project

Aspect	Mini Project (Year 3)	Final Year Project (Year 4)
Duration	1 semester	1–2 semesters
Chapters	5 (condensed)	5 (detailed)
Literature Review	2–4 sources	10+ sources, comparative
Testing	Basic	Unit, Integration, Acceptance
Defense	Informal/none	Formal panel defense
Documentation	Lightweight	Full (declaration, certification, appendices)